



Amazon API Gateway

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

OpenAPI



OpenAPI Specification (OAS) defines a standard, language-agnostic interface to RESTful APIs which allows both humans and computers to discover and understand the capabilities of the service without access to source code, documentation, or through network traffic inspection.

Swagger and OpenAPI used to be the same thing but as of OpenAPI V3, Swagger and OpenAPI are two different things

- OpenAPI = Specification
- Swagger = Tools for implementing the specification

OpenAPI can be represented as either **JSON** or **YAML**

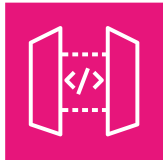
```
paths:
  /users:
    post:
      summary: Adds a new user
      requestBody:
        content:
          application/json:
            schema: # Request body contents
              type: object
              properties:
                id:
                  type: integer
                name:
                  type: string
              example:
                id: 10
                name: Andrew Brown
      responses:
        '200':
          description: OK
```

API Gateway

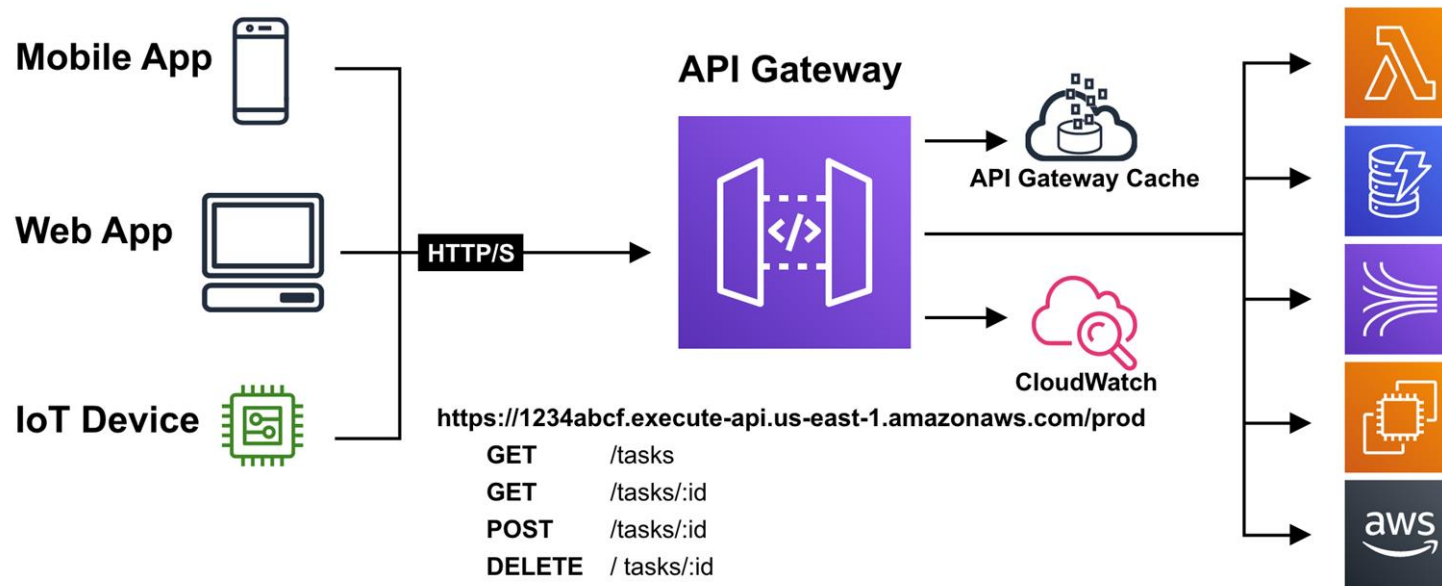
What is an API Gateway?

An API Gateway is a program that sits between a single-entry point and multiple backends.

API Gateway allows for throttling, logging, routing logic or formatting of the request and response



Amazon API Gateway is a solution for **creating secure APIs** in your cloud environment at **any scale**. Create APIs that act as a front door for applications to access data, business logic, or functionality from back-end services.



API Gateway

There are **three types of APIs** for API Gateway:

Rest API (API Gateway V1)

- Complete Control over Request and Response
- Most feature-rich
- Higher costs
- Both Private and Public API options

HTTP API (API Gateway V2)

- Low Latency
- Simpler feature set
- Low costs
- Only Public APIs

WebSockets API

- persistent connections for real-time use cases such as chat applications or dashboards

Import API via OpenAPI 3

For both REST and HTTP APIs you can **import an Open API 3 file** when creating your API.

```
openapi: 3.0.0
info:
  version: 1.0.0
  title: Simple API
  description: My API example
paths:
  /hello:
    get:
      summary: Returns a greeting message
      responses:
        '200':
          description: A greeting message
          content:
            application/json:
              schema:
                type: object
                properties:
                  message:
                    type: string
                    example: Hello, world!
```

```
AWSTemplateFormatVersion: '2010-09-09'
Resources:
  MyApi:
    Type: AWS::ApiGateway::RestApi
    Properties:
      Name: SimpleApi
      Body:
        'Fn::Transform':
          Name: 'AWS::Include'
          Parameters:
            Location: !Sub 's3://${BucketName}/openapi.yaml'
```



OpenAPI Extensions for AWS

AWS extends the OpenAPI definition so you can define AWS API Gateway specific features in your OpenAPI File.

x-amazon-apigateway-extension

- -any-method object
- -cors object
- -api-key-source property
- -auth object
- -authorizer object
- -authtype property
- -binary-media-types property
- -documentation object
- -endpoint-configuration object
- -gateway-responses object
- -gateway-responses.gatewayResponse object
- -gateway-responses.responseParameters object
- -gateway-responses.responseTemplates object
- -importexport-version
- -tag-value property
- -integration object
- -integrations object
- -integration.requestTemplates object
- -integration.requestParameters object
- -integration.responses object
- -integration.response object
- -integration.responseTemplates object
- -integration.responseParameters object
- -integration.tlsConfig object
- -minimum-compression-size
- -policy
- -request-validator property
- -request-validators object
- -request-validators.requestValidator object

OpenAPI Extensions for AWS

Here's an example of using the **-policy extension**
To define IAM Policy for specific API paths.



```
---
x-amazon-apigateway-policy:
  Version: '2012-10-17'
  Statement:
    - Effect: Allow
      Principal: "*"
      Action: execute-api:Invoke
      Resource:
        - execute-api/*
    - Effect: Deny
      Principal: "*"
      Action: execute-api:Invoke
      Resource:
        - execute-api/*
      Condition:
        IPAddress:
          aws:SourceIp: 192.0.2.0/24
```

OpenAPI Extensions for AWS

Here's an example of using the **-cors**
To define CORS for the API.



```
---
x-amazon-apigateway-cors:
  allowOrigins:
    - https://www.example.com
  allowCredentials: true
  exposeHeaders:
    - x-apigateway-header
    - x-amz-date
    - content-type
  maxAge: 3600
  allowMethods:
    - GET
    - OPTIONS
    - POST
  allowHeaders:
    - x-apigateway-header
    - x-amz-date
    - content-type
```

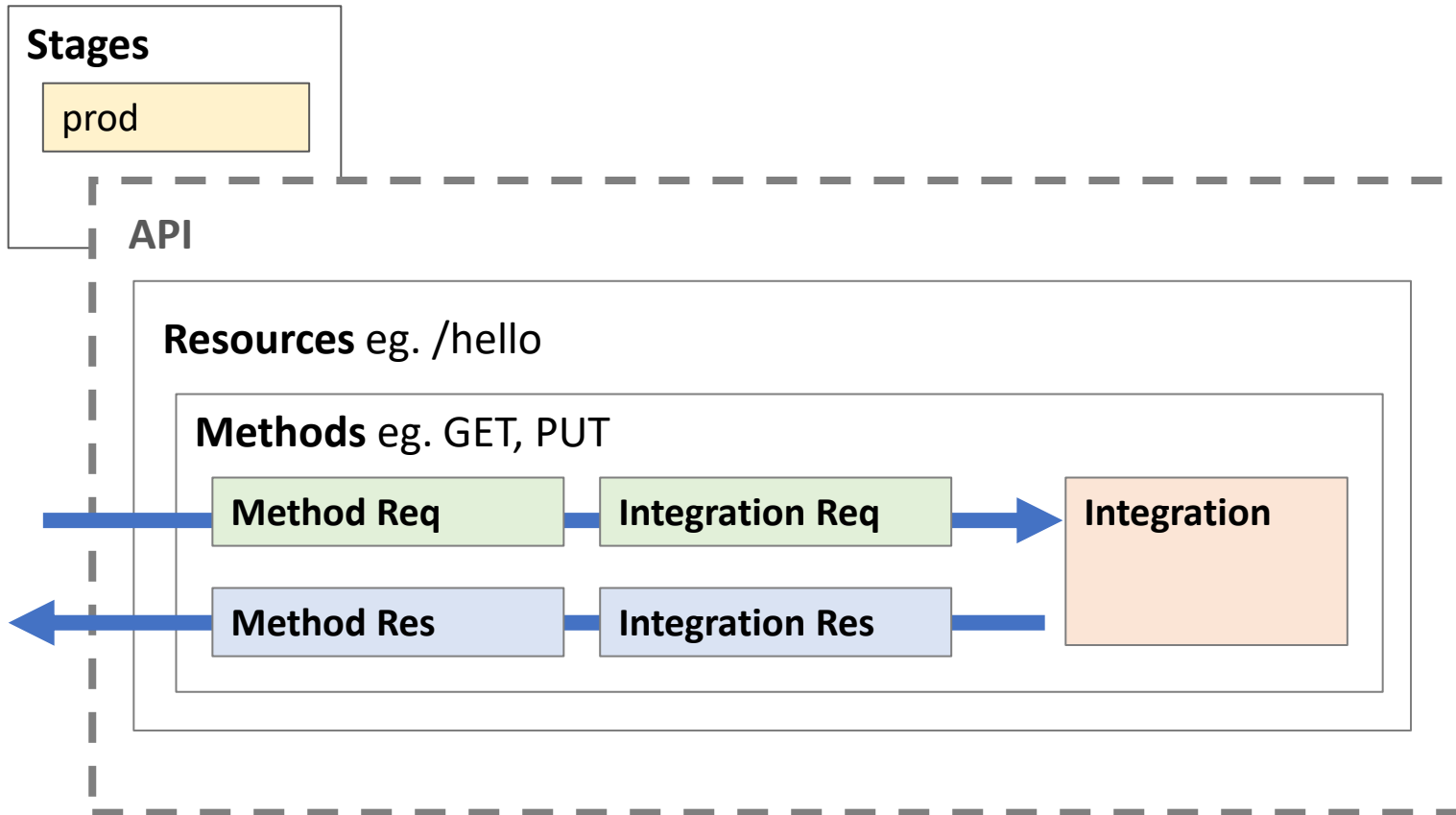
API Gateway REST vs HTTP

Endpoint types	Rest	HTTP	Authorization	Rest	HTTP	Development	Rest	HTTP
Regional	Yes	Yes	Resource Policies	Yes	No	Automatic deployments	No	Yes
Edge-optimized		No	IAM		Yes	User-controlled deploys	Yes	Yes
Private		Amazon Cognito	CORS Configuration					
	Custom Auth with an AWS Lambda function	Test Invocations	No	Yes	Caching	No		
Security	Yes	Yes			JWTs		Custom Gateway Response	
Mutual TLS authentication		No			API Management		Canary release deploys	
Certificates for backend auth		Custom Domains	Yes	No	Request Validation			
AWS WAF		API Keys	Yes	Yes	Request Body Transform			
	Per-client rate limiting	Request Param Transform			Yes			
	Per-client usage throttling							

API Gateway REST vs HTTP

Monitoring	Rest	HTTP	Integrations	Rest	HTTP
CloudWatch Metrics	Yes	Yes	Mock Integrations	Yes	No
CloudWatch Logs			Public HTTP endpoints		Yes
Amazon Data Firehose		No	AWS Services		
Execution Logs			AWS Lambda functions		
AWS X-Ray Tracing			Private integrations with NLB	No	
			Private integrations with ALB		
			Private integrations with Cloud Map		

API Gateway REST Components



API — container for multiple resources

Resources — represent an endpoint.
Resources are nested within other resources

Methods — are individual methods for a specific endpoint. Methods have allow you to customize Requests and Response

- Method Request
- Integration Request
- Method Response
- Integration Response

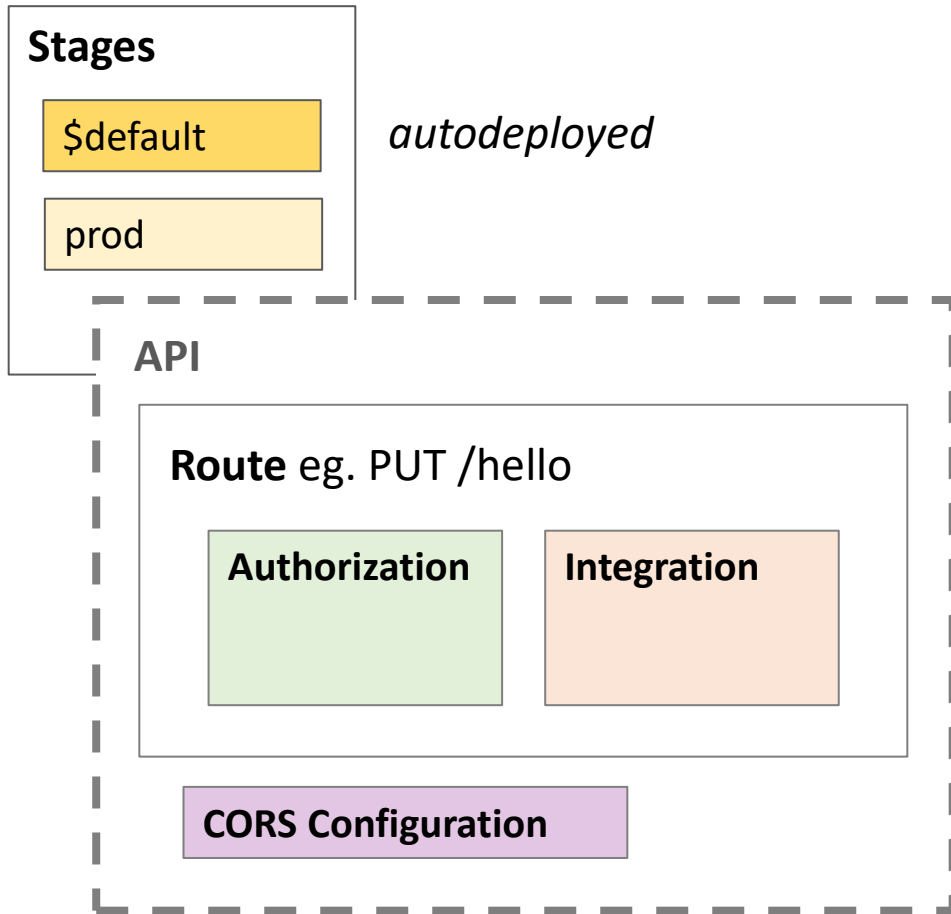
Integration

The integration service that will be called

- Lambda function (AWS_Proxy)
- HTTP
- Mock
- AWS Service
- VPC Link

Stage — Versions of your API
Stages must be deployed to be accessible.

API Gateway HTTP Components



API — container for multiple routes

Routes — represent an endpoint.
routes are nested within other routes

- You choose the method
- You defined your endpoint

Integration

The integration service that will be called

- Lambda function (AWS_PROXY)
- HTTP
- AWS Service (limited to specific services)
 - EventBridge, SQS, AppConfig, Kinesis Data Streams, Step Functions
- VPC Link

Stage — Versions of your API

Stages must be deployed to be accessible.

API Gateway has a special stage called \$default.

All changes that you make to your API are autodeployed to that stage.

API Gateway Data Models

API Gateway Data Model can be used to help **validate and enforce the expected data structure** of incoming requests

Data Model **schema**

```
ItemsModel:
  Type: AWS::ApiGateway::Model
  Properties:
    RestApiId: !Ref MyApi
    Name: ItemsModel
    ContentType:
application/json
    Schema:
      type: object
      properties:
        id:
          type: string
        name:
          type: string
        description:
          type: string
        price:
          type: number
      required:
        - id
        - name
        - price
```

```
PostMethod:
  Type: AWS::ApiGateway::Method
  Properties:
    RestApiId: !Ref MyApi
    ResourceId: !GetAtt ApiResource.Id
    HttpMethod: POST
    AuthorizationType: NONE
    RequestModels:
      application/json: !Ref ItemsModel
```

API Gateway Data Models

Example of **json payload**

```
{
  "id": 1,
  "type": "dog",
  "price": 249.99
}
```

Example of data model schema in **json format**

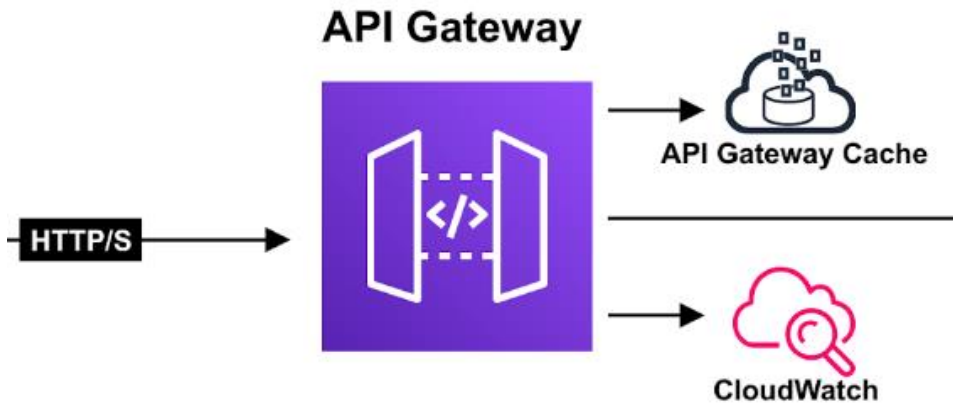
```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "PetStoreModel",
  "type": "object",
  "required": [ "price", "type" ],
  "properties": {
    "id": {
      "type": "integer"
    },
    "type": {
      "type": "string",
      "enum": [ "dog", "cat", "fish" ]
    },
    "price": {
      "type": "number",
      "minimum": 25.0,
      "maximum": 500.0
    }
  }
}
```

API Gateway REST – Endpoint Comparison

	Regional Endpoint	Edge-Optimized Endpoint	Private Endpoint
Purpose	In-region clients	Global clients	Access within VPC
Endpoint URL	<api-id>.execute-api.<region>.amazonaws.com	<api-id>.execute-api. .amazonaws.com	<api-id>.execute-api.<region>.vpce.amazonaws.com
Use Case	Low-latency regional access	Global access with low latency	Secure internal access
Latency	Low within the same region	Low globally via CloudFront	Low within VPC
Cost	Standard API Gateway pricing	Additional CloudFront costs	Standard pricing, no CloudFront costs
Security	Standard (IAM, Cognito)	Standard (IAM, Cognito)	Enhanced via VPC isolation
Network Access	Public internet	Public internet, optimized via CloudFront	Within VPC only
Deployment	Simple regional setup	Includes CloudFront distribution setup	Requires VPC endpoint configuration
Scalability	Automatic within the region	Automatic via CloudFront	Scales within VPC resources

API Gateway REST- Caching

API Caching can be enabled to cache your endpoint's response to API calls.



- When enabled on a stage, API Gateway caches responses from your endpoint for a specified time-to-live (TTL) period.
- API Gateway responds to requests by looking up the response from the cache. (instead of making a request to the endpoint)
- Reduces the number of calls made to your endpoint.
- Improves latency of the requests made to your API.

Enable caching at **stage-level**

```
aws apigateway update-stage \  
--rest-api-id $api_id \  
--stage-name dev \  
--patch-operations  
op=replace,path=/cacheClusterEnabled,value=true  
op=replace,path=/cacheClusterSize,value=0.5
```

Enable caching at **method-level**

```
aws apigateway update-method \  
--rest-api-id $api_id \  
--resource-id $resource_id \  
--http-method GET \  
--patch-operations  
op=replace,path=/methodSettings/*/ca  
chingEnabled,value=true
```

API Gateway REST- Caching

You can invalidate the cache by **turning the cache off and then back on.**

```
aws apigateway update-stage \  
--rest-api-id abc123 \  
--stage-name dev \  
--patch-operations op=replace,path=/cacheClusterEnabled,value=false  
  
aws apigateway update-stage \  
--rest-api-id abc123 \  
--stage-name dev \  
--patch-operations op=replace,path=/cacheClusterEnabled,value=true
```



API Gateway REST API - Throttling

There are four basic throttling for API Gateway REST API

- AWS throttling limits
 - applied across all accounts and clients in a region
- Per-account limits (Applies to REST, HTTP and Web Sockets)
 - applied to all APIs in per account per region
 - 10,000 requests per second (RPS) with an additional burst capacity
- Per-API
 - Per stage level applied at the API method level for a stage
 - Usage plans lets you set throttling at the per-method level.
- Per-client throttling limits
 - applied to clients that use API keys associated with your usage plan as client identifier.

When throttling in affect requests with return a **429 Too Many Requests** error response

API Gateway REST API Usage Plan

A **usage plan** specifies who can access one or more deployed API stages and methods—and sets the target request rate to start throttling requests

Throttle Settings:

- Burst Limit: The max number of requests that API Gateway can serve concurrently in a short period.
- Rate Limit: The steady-state request rate (requests per second) that API Gateway can serve.

Quota Settings:

- Limit: The max number of requests that can be made
- Period: The time period for the quota (e.g day, week, month).

API Keys:

The identifier of the API key to be associated with the usage plan.

API keys are created with API Gateway REST UI.

These are not the same as AWS Credentials

```
ApiUsagePlan:
  Type: AWS::ApiGateway::UsagePlan
  Properties:
    UsagePlanName: SimpleApiUsagePlan
    Description: Usage plan for Simple API
    Quota:
      Limit: 1000
      Period: MONTH
    Throttle:
      BurstLimit: 200
      RateLimit: 100
    ApiStages:
      - ApiId: !Ref MyApi
        Stage: v1
```

Per method throttling can be set in a Usage Plan, its not demonstrated in the CFN template above

API Gateway HTTP API - Throttling

There are two basic throttling for API Gateway HTTP API

- Account-Level throttling per Region
 - across all APIs within an AWS account, per Region.
 - 10,000 requests per second (RPS) with an additional burst capacity (can be increased)
- **Route-Level throttling** 
 - override the account-level request throttling limits for a specific stage or for individual routes in your API.

```
aws apigatewayv2 update-stage \  
  --api-id a1b2c3d4 \  
  --stage-name dev \  
  --route-settings '  
{  
  "GET /pets {  
    "ThrottlingBurstLimit":100,  
    "ThrottlingRateLimit":2000  
  }  
'
```

When throttling in affect requests with return a **429 Too Many Requests** error response

API Gateway REST – CloudWatch Logging

You can **configure** API Gateway REST to log to CloudWatch Logs

```
TestStage:
  Type: AWS::ApiGateway::Stage
  Properties:
    StageName: test
    RestApiId: !Ref MyAPI
    DeploymentId: !Ref Deployment
    Description: "test stage description"
    MethodSettings:
      - ResourcePath: "/*"
        HttpMethod: "*"
        LoggingLevel: INFO
    AccessLogSetting:
      DestinationArn: !GetAtt MyLogGroup.Arn
      Format: $context.extendedRequestId
        $context.identity.sourceIp
        $context.identity.caller $context.identity.user
        [$context.requestTime] "$context.httpMethod
        $context.resourcePath $context.protocol"
        $context.status $context.responseLength
        $context.requestId
  MyLogGroup:
    Type: AWS::Logs::LogGroup
    Properties:
      LogGroupName: !Join
        - '-'
        - !Ref MyAPI
        - access-logs
```

Example log data

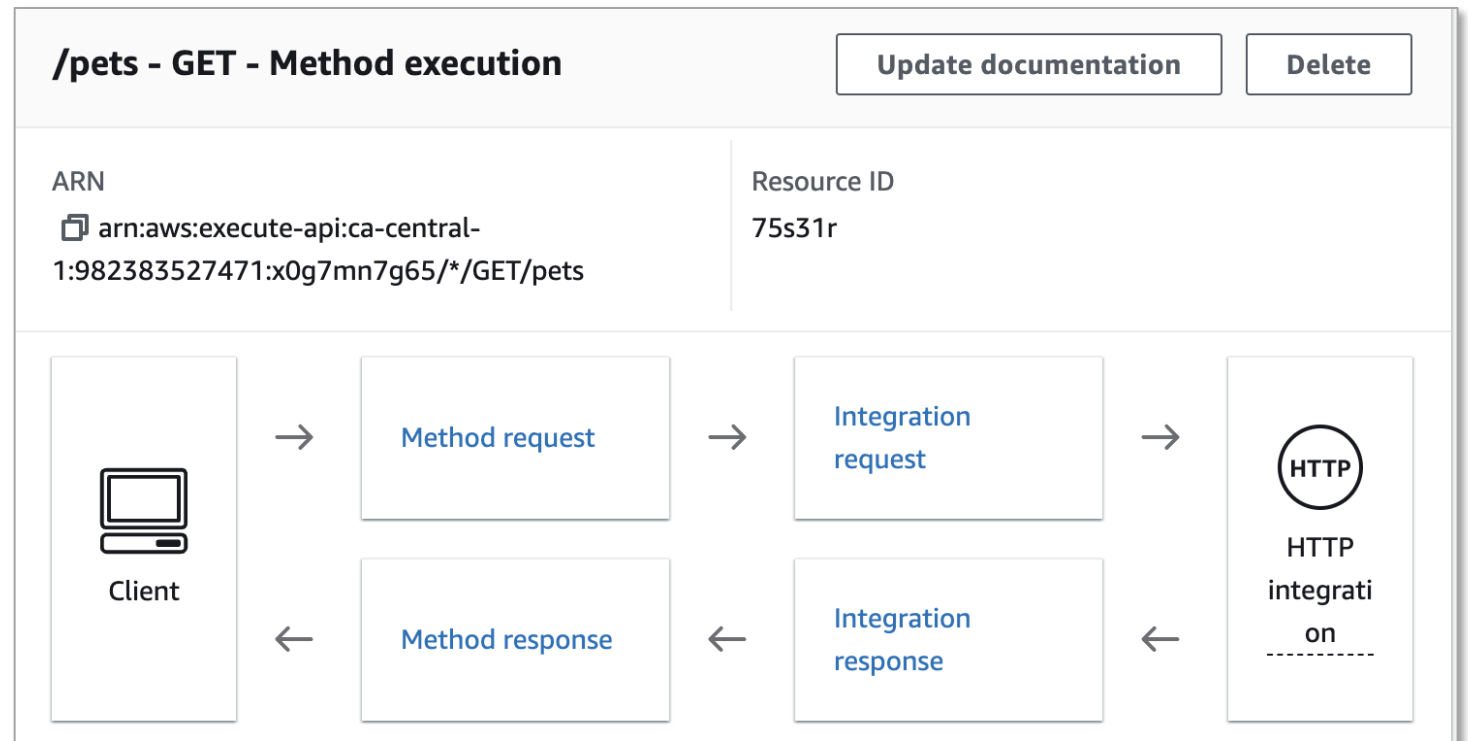
```
RequestId: abcd1234-5678-90ab-cdef-EXAMPLE
RequestTime: 02/Jan/2024:12:00:00 +0000
HTTPMethod: GET ResourcePath: /myresource
Status: 200 Protocol: HTTP/1.1 ResponseLength: 123

Request:
{
  "resource": "/myresource",
  "path": "/myresource",
  "httpMethod": "GET",
  "headers": {
    "accept": "application/json",
    "Host": "abc123.execute-api.us-east-1.amazonaws.com",
    "User-Agent": "curl/7.64.1",
    "X-Amzn-Trace-Id": "Root=1-5f2a39d4-9c10a8c0e7c0339330fa2bc2",
    "X-Forwarded-For": "203.0.113.1",
    "X-Forwarded-Port": "443",
    "X-Forwarded-Proto": "https"
  },
  "queryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "resourceId": "abc123",
    "resourcePath": "/myr
```

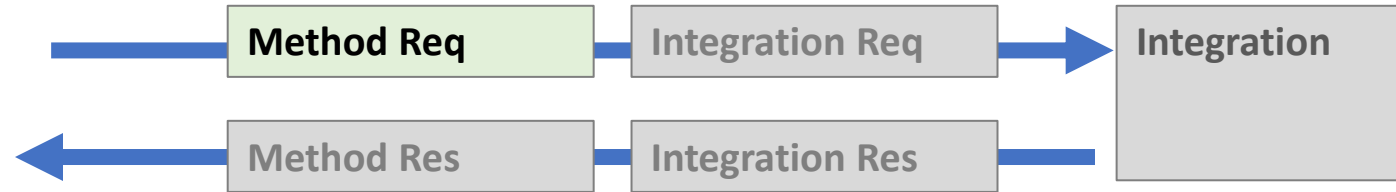
API Gateway REST - Method Configuration

API Gateway REST allows us to configure the following for a method:

- Method request
- Integration request
- Integration response
- Method response



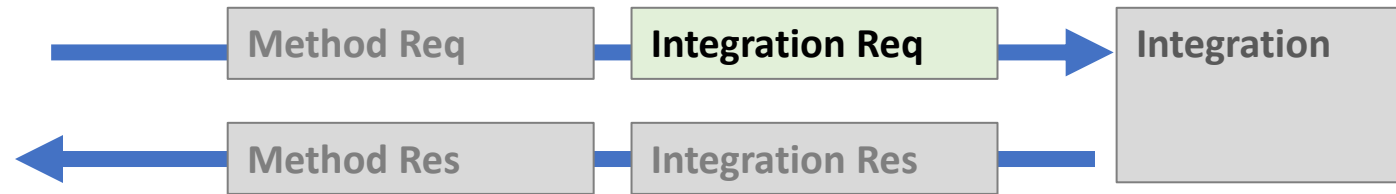
API Gateway REST – Method Configuration



Method Request Configuration Options

- **Authorization:**
 - None — No authorization required.
 - IAM — AWS IAM policies control access.
 - Cognito User Pools — Authorization through Amazon Cognito.
 - Lambda Authorizer — Custom authorization using a Lambda function.
- **API Key Required** — Whether an API Gateway API key is required for the method.
- **Request Parameters:**
 - Path Parameters — Parameters embedded in the resource path.
 - Query String Parameters — Parameters in the query string of the URL.
 - Header Parameters — HTTP headers included in the request.
- **Request Models** — Model schemas for validating the request payload (JSON Schema).

API Gateway REST – Method Configuration

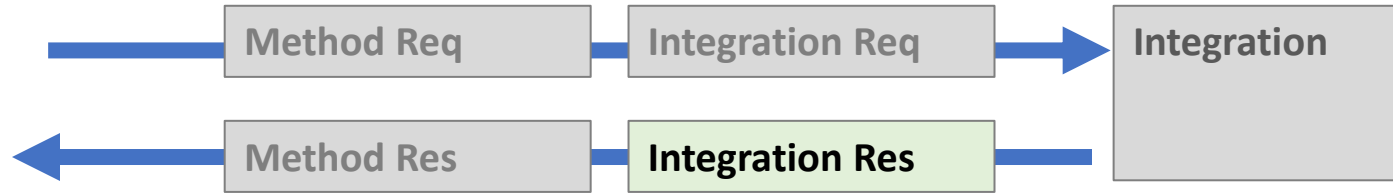


Integration Request

- **Integration Type:**
 - HTTP / HTTP_PROXY — Integrates with an HTTP endpoint. (there is a proxy variant)
 - AWS / AWS_PROXY — Integrates with an AWS service or a Lambda function. (there is a proxy variant)
 - Mock — Returns a mock response without calling any backend.
- **Integration URI:** The endpoint URI for the integration (for HTTP or AWS integrations).
- **HTTP Method:** The HTTP method used for the integration request (for HTTP integrations).
- **Request Templates:** Mapping templates to transform the incoming request data (e.g., from JSON to another format).
- **Request Parameters:**
 - **Path Parameters:** Mappings for path parameters.
 - **Query String Parameters:** Mappings for query string parameters.
 - **Header Parameters:** Mappings for HTTP headers.

Configuration options will greatly vary based on Integration Types

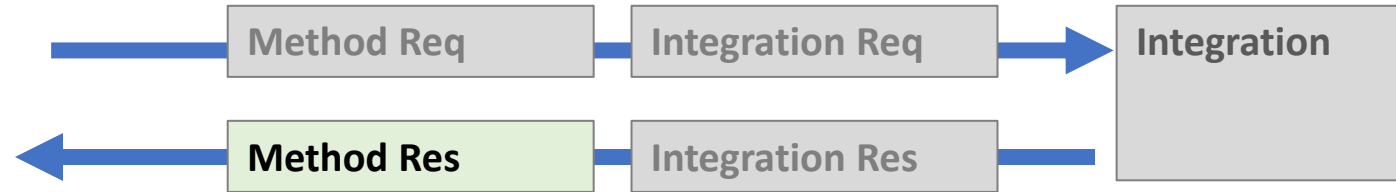
API Gateway REST – Method Configuration



Integration Response

- **Returned Status Code:** The status code returned by the backend integration.
- **Response Templates:** Mapping templates to transform the response data (e.g., from JSON to another format).
- **Header Mappings:** Mappings to set or modify HTTP headers in the response.
- **Content Handling:**
 - Passthrough
 - Convert to binary
 - Convert to Text

API Gateway REST – Method Configuration



Method Response

- **Status Code:** The status code returned by the method.
- **Response Models:** Model schemas for validating the response payload (JSON Schema).
- **Response Parameters:**
 - **Header Mappings:** Mappings to add or modify HTTP headers in the response.

API Gateway REST – AWS Integration

When using the **AWS** Integration type you can send requests directly to AWS Services.

You can send requests to **100+ AWS Services**.

Notice that you send a **specific API Action**

You will need to look up that API action to determine what parameters need to be sent to that API action.

```
GetItemMethod:
  Type: AWS::ApiGateway::Method
  Properties:
    AuthorizationType: NONE
    HttpMethod: GET
    ResourceId: !Ref ApiResource
    RestApiId: !Ref MyApi
    Integration:
      IntegrationHttpMethod: POST
      Type: AWS
      Uri: !Sub arn:aws:apigateway:${AWS::Region}:dynamodb:action/GetItem
      Credentials: !GetAtt ApiGatewayDynamoDBRole.Arn
      RequestTemplates:
        application/json: |
          {
            "TableName": "MyDynamoDBTable",
            "Key": {
              "id": {
                "S": "${input.params('id')}"
              }
            }
          }
      }
```

AWS_PROXY is specifically for Lambda, you can also use AWS for Lambda but it not act the same way as AWS_PROXY

API Gateway REST – AWS Proxy

AWS_PROXY is also known as **Lambda proxy integration** skips Integration Request and Integration Response and directly send the request to AWS Lambda.

```
GetItemMethod:
  Type: AWS::ApiGateway::Method
  Properties:
    AuthorizationType: NONE
    HttpMethod: GET
    ResourceId: !Ref ApiResource
    RestApiId: !Ref MyApi
    Integration:
      IntegrationHttpMethod: POST
      Type: AWS_PROXY
      Uri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${MyLambdaFunction.Arn}/invocation
```

The AWS Lambda directly handles the headers, parameters, and the Lambda has to return an HTTP response and any additional headers (include CORS headers)

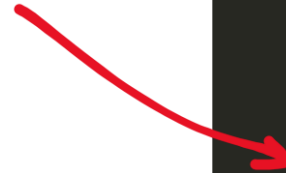
API Gateway REST – AWS Proxy

The Lambda used in the Lambda Proxy has to return an HTTP request

```
MyLambdaFunction:
  Type: AWS::Lambda::Function
  Properties:
    Handler: index.handler
    Role: !GetAtt LambdaExecutionRole.Arn
    Runtime: python3.8
    Code:
      ZipFile: |
        def handler(event, context):
            id = event['queryStringParameters']['id']
            return {
                'statusCode': 200,
                'body': f'{{"message": "Item with ID {id} retrieved"}}',
                'headers': {
                    'Content-Type': 'application/json'
                }
            }
```

API Gateway REST – HTTP

HTTP also known as **HTTP custom integration** allows you to send your requests to an API endpoint to externally integrate third-party applications



HTTP_PROXY is the same as an HTTP Integration except that it skips Integration Request and Integration Response and directly send the request to the app

```
GetItemMethod:
  Type: AWS::ApiGateway::Method
  Properties:
    AuthorizationType: NONE
    HttpMethod: GET
    ResourceId: !Ref ApiResource
    RestApiId: !Ref MyApi
    Integration:
      IntegrationHttpMethod: GET
      Type: HTTP
      Uri: "https://jsonplaceholder.typicode.com/todos/1"
      PassthroughBehavior: WHEN_NO_MATCH
      RequestParameters:
        integration.request.querystring.userId: method.request.querystring.userId
      IntegrationResponses:
        - StatusCode: 200
          ResponseTemplates:
            application/json: |
              #set($inputRoot = $input.path('$'))
              {
                "userId": "$inputRoot.userId",
                "id": "$inputRoot.id",
                "title": "$inputRoot.title",
                "completed": "$inputRoot.completed"
              }
        - StatusCode: 400
        - StatusCode: 500
      RequestParameters:
        method.request.querystring.userId: false
      MethodResponses:
        - StatusCode: 200
          ResponseModels:
            application/json: Empty
        - StatusCode: 400
        - StatusCode: 500
```

API Gateway REST – Mock

Mock allow you to returned canned/faked responses. Mock requests are useful when you need a placeholder endpoint, the data returned is static and never changes, or you need a mock API endpoint to be used against a test suite.

```
MockMethod:
  Type: AWS::ApiGateway::Method
  Properties:
    AuthorizationType: NONE
    HttpMethod: GET
    ResourceId: !Ref ApiResource
    RestApiId: !Ref MyApi
    Integration:
      Type: MOCK
      RequestTemplates:
        application/json: '{ "statusCode": 200 }'
    MethodResponses:
      - StatusCode: 200
        ResponseModels:
          application/json: Empty
        ResponseParameters:
          method.response.header.Content-Type: false
      - StatusCode: 400
      - StatusCode: 500
    IntegrationResponses:
      - StatusCode: 200
        ResponseTemplates:
          application/json: |
            '{ "message": "This is a mock response" }'
        ResponseParameters:
          method.response.header.Content-Type: |
            "'application/json'"
      - StatusCode: 400
      - StatusCode: 500
```

API Gateway REST – Deployment

You cannot use an API until it has been **deployed to a stage**.



```
aws apigateway create-deployment --rest-api-id $API_ID --stage-name $STAGE_NAME
```

The deployment will be available at endpoint with a domain name owned by AWS

```
https://{restapi-id}.execute-api.{region}.amazonaws.com/{stageName}/your-resource
```

This endpoint format will vary depending on the API Endpoint type eg. regional, Edge, Private

You can configure a custom domain with API Gateway REST

API Gateway HTTP Integrations

AWS_PROXY is **AWS Lambda Integration** which will send requests to Lambda.

API Gateway HTTP will not apply CORS to a response so if you need CORS you'll need to return the headers.

```
MyLambdaIntegration:
  Type: AWS::ApiGatewayV2::Integration
  Properties:
    ApiId: !Ref MyHttpApi
    IntegrationType: AWS_PROXY
    IntegrationUri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${MyLambdaFunction.Arn}/invocations
    PayloadFormatVersion: '2.0'
```



```
MyEventBridgeIntegration:
  Type: AWS::ApiGatewayV2::Integration
  Properties:
    ApiId: !Ref MyHttpApi
    IntegrationType: AWS
    IntegrationSubtype: EventBridge-PutEvents
    CredentialsArn: !GetAtt MyEventBridgeRole.Arn
    RequestParameters:
      Source: "api-gateway"
      DetailType: "Request Body"
      Detail: "$request.body"
    PayloadFormatVersion: '1.0'
```



AWS integration is for very specific AWS services:

- **EventBridge**
- SQS
- AppConfig
- Kinesis Data Streams
- Step Functions

API Gateway CORS

CORS allows a web page to access restricted resources from a server on a domain different than the domain that served the web page.

A cross-origin HTTP request is one that is made to:

- A different domain (for example, from example.com to amazondomains.com)
- A different subdomain (for example, from example.com to petstore.example.com)
- A different port (for example, from example.com to example.com:10777)
- A different protocol (for example, from https://example.com to http://example.com)

API Gateway CORS

You can configure CORS headers for both REST API and HTTP API

	REST API	HTTP API
CORS Setup	Manual setup required	Simplified setup
Configuration	Granular control	Less granular
CORS	No default CORS	Easy console setup
Integration	Lambda, HTTP, Mock	Lambda, HTTP Proxy
Complexity	More complex	Easier and faster
Security	Detailed settings	Adequate for most needs
Use Cases	Detailed requirements	Common needs, serverless

API Gateway REST – CORS Non-Integration Proxy

For API Gateway when you are not using a Lambda Integration Proxy you can tell API Gateway to return CORS headers

You can specify this down-to the **method-level**

```
MyMethod:
  Type: 'AWS::ApiGateway::Method'
  Properties:
    AuthorizationType: 'NONE'
    HttpMethod: 'POST'
    ResourceId:
      Ref: 'MyResource'
    RestApiId:
      Ref: 'MyApi'
    Integration:
      IntegrationHttpMethod: 'POST'
      Type: 'MOCK'
      RequestTemplates:
        application/json: '{ "statusCode": 200 }'
      IntegrationResponses:
        - StatusCode: 200
          ResponseTemplates:
            application/json: '"Hello"'
          ResponseParameters:
            method.response.header.Access-Control-Allow-Origin: "'*'"
            method.response.header.Access-Control-Allow-Headers: "'Content-Type'"
            method.response.header.Access-Control-Allow-Methods: "'POST,OPTIONS'"
      MethodResponses:
        - StatusCode: 200
          ResponseModels:
            application/json: 'Empty'
          ResponseParameters:
            method.response.header.Access-Control-Allow-Origin: false
            method.response.header.Access-Control-Allow-Headers: false
            method.response.header.Access-Control-Allow-Methods: false
```

API Gateway REST – CORS Non-Integration Proxy

You need to define an OPTIONS method for
your endpoint for a **CORS Preflight check**

```
OptionsMethod:
  Type: 'AWS::ApiGateway::Method'
  Properties:
    AuthorizationType: 'NONE'
    HttpMethod: 'OPTIONS'
    ResourceId:
      Ref: 'MyResource'
    RestApiId:
      Ref: 'MyApi'
    Integration:
      IntegrationResponses:
        - StatusCode: 200
          ResponseParameters:
            method.response.header.Access-Control-Allow-Origin: "'*'"
            method.response.header.Access-Control-Allow-Headers: "'Content-Type'"
            method.response.header.Access-Control-Allow-Methods: "'POST,OPTIONS'"
          ResponseTemplates:
            application/json: '{}'
```



```
      RequestTemplates:
        application/json: '{ "statusCode": 200 }'
      Type: 'MOCK'
    MethodResponses:
      - StatusCode: 200
        ResponseParameters:
          method.response.header.Access-Control-Allow-Origin: false
          method.response.header.Access-Control-Allow-Headers: false
          method.response.header.Access-Control-Allow-Methods: false
```

API Gateway REST – Binary Media Types

API Gateway REST can be configured to **handle binary media types**.

Binary types is anything not plain text eg. JPG, ZIP, PDF, MP4

Non-proxy Binary Type Integrations

You need to **update the supported binary types list**

```
aws apigateway update-rest-api --rest-api-id abc123 \  
# different examples, these would need to be run in separate API calls  
--patch-operations op=add,path=/binaryMediaTypes/image~1png  
--patch-operations op=add,path=/binaryMediaTypes/application~1octet-stream  
--patch-operations op=add,path=/binaryMediaTypes/image~1jpeg  
--patch-operations op=add,path=/binaryMediaTypes/audio~1mpeg
```

AWS Lambda Proxy Binary Type Configuration

- You need to update supported binary type lists
- AND you must **base64-encode** your function's response.

```
require 'base64'  
require 'json'  
  
def lambda_handler(event:, context:)  
  file_path = '/path/to/your/binary/file.jpg'  
  begin  
    binary_data = File.binread(file_path)  
    base64_data = Base64.encode64(binary_data)  
    {  
      statusCode: 200,  
      isBase64Encoded: true,  
      headers: {'Content-Type' => 'image/jpeg'},  
      body: base64_data  
    }  
  rescue => e  
    {  
      statusCode: 500,  
      body: JSON.generate({ error: e.message })  
    }  
  end  
end
```

API Gateway REST – Resource Policy

```
aws apigateway update-rest-api \  
--rest-api-id abc123 \  
--patch-operations op=add,path=/policy,value=file://policy.json
```

API Gateway REST lets you set a **resource policy**.
You can restrict or allow access for very specific API
endpoints

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Effect": "Allow",  
    "Principal": {  
      "AWS": [  
        "arn:aws:iam::123456789012:root",  
        "arn:aws:iam::123456789012:user/andrewbrown",  
        "arn:aws:iam::123456789012:role/dev"  
      ]  
    },  
    "Action": "execute-api:Invoke",  
    "Resource": [  
      "execute-api:/*/*/*"  
    ]  
  }]  
}
```

API Gateway REST – CORS Lambda Integration Proxy

For a Lambda proxy integration or HTTP proxy integration, **your backend is responsible for returning the CORS Headers** because a proxy integration doesn't return an integration response.

```
import json

def lambda_handler(event, context):
    return {
        'statusCode': 200,
        'headers': {
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Origin': 'https://www.example.com',
            'Access-Control-Allow-Methods': 'OPTIONS,POST,GET'
        },
        'body': json.dumps('Hello from Lambda!')
    }
```



API Gateway HTTP – CORS Configuration

API Gateway HTTP API can set CORS.

CORS Headers can be configured but **applies to all endpoints** in the API

```
aws apigatewayv2 update-api \  
--api-id api-id \  
--cors-configuration AllowOrigins="https://www.example.com"
```



CORS Headers you can set for HTTP API

- Access-Control-Allow-Origin (allowOrigin)
- Access-Control-Allow-Credentials (allowCredentials)
- Access-Control-Expose-Headers (exposeHeaders)
- Access-Control-Max-Age (maxAge)
- Access-Control-Allow-Methods (allowMethods)
- Access-Control-Allow-Headers (allowHeaders)

API Gateway HTTP – Parameter Mapping

Parameter Mapping allows you to **modify requests and responses** **from integrations** before API Gateway returns the response to clients.

You can change:

- **Headers** (Requests or Responses) — append|overwrite|remove:header.headername
- **Query String** (Requests)— append|overwrite|remove:querystring.querystring-name
- **Paths** (Requests) — overwrite:path
- **Status Code** (Responses) — overwrite:statusCode

You can access dynamic values:

- \$request
- \$response
- \$context
- \$stageVariables

Example of parameter mapping

- Set Header1 to new Header2
- Clear value of Header1



```
aws apigatewayv2 create-integration \  
--api-id abcdef123 \  
--integration-type HTTP_PROXY \  
--payload-format-version 1.0 \  
--integration-uri 'https://api.example.com' \  
--integration-method ANY \  
--request-parameters '{  
  "append:header.header2": "$request.header.header1",  
  "remove:header.header1": ""  
}'
```

There are many Reserved headers that cannot be changed eg. Authorization, Origin, Forwarded

API Gateway – Access Controls

REST API

HTTP API

IAM authorizers — provide a SigV4 to sign requests

Lambda authorizers — Implement your own custom Lambda for authorization

(REST API would need to implement a lambda authorizer to do JWT)

JWT authorizers

Directly configure a JWT authorizer.

Resource policies

JSON document policy you can attach to your API (think similar to bucket policy)

IAM tags — Use IAM policy along with tags

Amazon Cognito user pools

Authorize directly with Cognito

Endpoint policies for interface VPC endpoints

Resource policy for private APIs with interface endpoint